

Grounded Q&A over Enterprise Documents

Avathon AI/ML Hiring Challenge — Track D (RAG / LLM Knowledge Systems) × Scenario S2 · Raushan Kumar · github.com/Raushan0502/avathon-rag-s2

Every figure below regenerates with `python scripts/report_metrics.py`, which reads them from `results/` rather than from prose.

1. Problem & Domain

A compliance officer or in-house counsel needs a specific fact out of a corpus that is large, constantly changing, and legally consequential — *does our insider-trading policy cover a spouse's holdings?* Today that is manual: open the filing, use the table of contents, read. The answer must be **grounded and citable**: in a regulated setting a fabricated disclosure is worse than none.

The dataset is real, not synthetic — 100 public documents from the SEC EDGAR API, NIST, and the public Enron/AESLC email corpus, spanning the four content types S2 names across **three file formats**:

Type	n	Format	Source
Reports — 10-K / 8-K	20	HTML	SEC EDGAR
Contracts — EX-10 exhibits	20	HTML	SEC EDGAR
Policies — EX-19 / EX-97 exhibits	20	HTML	SEC EDGAR
Standards — NIST publications	20	PDF	NIST
Email — business correspondence	20	TXT	AESLC / Enron

Twelve issuers across seven industries, so questions span genuinely different business content rather than near-duplicate filings; each document is pinned in a manifest by `source_url` and `sha256`. The formats were chosen to force real extraction problems — HTML financial tables, PDF layouts, email headers — rather than one clean source.

Why Track D, not the others. RAG fits because the requirement is *auditability*: retrieval returns a passage, the answer cites it, a reader follows it back. Fine-tuning (B) bakes a snapshot into weights and cannot cite a source — and practically needs a GPU I do not have plus training cycles measured in hours, which inside 48 hours means one or two shots with no room to be wrong. Optimisation (C) has no decision variable or constraint set here. Agents (A) add non-deterministic control flow — the hardest thing to debug under time pressure — around a capability that is still retrieval.

2. Approach & Algorithm Decisions

`extract` → `normalise` → `validate` → `chunk` → `embed` → `index` → `retrieve` → `generate`

Chunking — token-budgeted, structure-preserving; 360 tokens, 60 overlap. Split on the document's own headings, never split a table row, prepend a document › section header to the embed text. *Rejected*: fixed-size windows (cut mid-table, and lose the context needed to tell twenty different "Item 7"s apart); one-chunk-per-section (10-K Item 7 exceeds any context window); semantic splitting (cost scales with corpus, gain unproven on filings that already carry headings). *Lost at boundaries*: long-range coreference — a chunk opening "These amounts exclude..." loses its antecedent — and cross-section reasoning, where a question needing Item 7 *and* Item 8 needs two chunks retrieved together. The budget is enforced: an earlier build silently truncated 80 chunks past the encoder's 512-token limit; an oversize split brought that to **0**.

Vector store — FAISS IndexFlatIP. Exact cosine, benchmarked against IVFFlat (40× faster, recall 1.000), HNSW (204×, recall 0.547 untuned) and IVFPQ (155×, 30× less memory). At 9,409 chunks exact search costs 8.67 ms, so approximation buys speed I do not need and costs recall I do. Flat wins *here*; at 10M chunks it would not.

Embedding — bge-small-en-v1.5 (384d), compared empirically against bge-large (1024d) rather than chosen by leaderboard position:

Model	MRR	R@k	P@k	Attainment	Query
bge-small	0.948	0.978	0.554	75%	64 ms
bge-large	0.940	0.978	0.604	82%	531 ms

A genuine trade: large is *better on precision*, marginally worse on MRR (−0.008, roughly noise), and costs **8.3× more per query** plus ~3.3 h to index this corpus on CPU. Small ships **on cost, not on quality** — on

GPU I would take large. Caveat: run on a 200-chunk subset (all golds plus distractors), because a full bge-large pass ran 10.7 h and was abandoned; sound as a *relative* comparison, not as absolute performance.

Retrieval — dense, BM25, and Reciprocal Rank Fusion (k=60), measured:

Mode	P@k	R@k	MRR	Attainment
dense	0.378	0.889	0.781	50.2%
bm25	0.208	0.733	0.555	27.6%
hybrid (RRF)	0.300	0.867	0.735	39.9%

Hybrid lost to dense — the opposite of the expected result, and of what an earlier draft of this README asserted before the numbers were in. The cause is **corpus-wide IDF dilution**: as the corpus grew 6 → 100 documents, the terms that discriminate in filings ("revenue", "Item 7", "Company") appeared everywhere, BM25's IDF flattened, and equal-weight RRF dragged the stronger ranker down. RRF is kept behind a flag as the right default on a lexically diverse corpus; dense ships here because that is what the measurement supports.

3. Results & Error Analysis

45 held-out questions, tiered by $k = 1/5/10/20$ to match real answer-set size. Precision is **structurally capped** — a third have single-chunk gold sections, so P@5 cannot exceed 0.2 for them — hence attainment ($P@k \div \text{ceiling } 0.752$) and MRR as headline metrics rather than raw precision.

Hallucination prevention and detection. The prompt constrains the model to retrieved context, requires a bracketed citation per claim, and permits refusal. Every answer is parsed and annotated: **41 cited, 4 refused, 0 UNGROUNDED**. The refusals are correct behaviour — retrieval genuinely missed (hit@k 40/45) and the model declined rather than inventing.

But cited ≠ correct, the central finding. Grounding says an answer pointed at context, not that it is right, so correctness is scored separately: lexical key-fact recall **0.567**, lexical accuracy **48.8%**, LLM judge **58.5% CORRECT**, 85.4% CORRECT+PARTIAL. **All 6 WRONG answers are properly grounded** — they cite real context and are still incorrect. Faithfulness alone is therefore an insufficient safety metric.

Where it fails. (i) The $k=1$ tier scores P@k 0.000 against MRR 0.278 — golds sit at ranks 2–3, and for $k=1$ the rank-1 hit is a *different* Apple RSU exhibit whose clause is verbatim-identical to the gold; no retriever can separate them, a corpus property rather than a ranking fault. (ii) Cross-section questions remain unsolved. (iii) The scorers disagree by ~10 points because the lexical one punishes paraphrase.

4. Production & Limitations

Production — 1,000 concurrent queries. The bottleneck is **not** FAISS: exact search is 8.67 ms and the 14 MB index sits in RAM on one node. It is (a) query embedding at 148 ms p50 single-threaded on CPU (~7 queries/s/core), and (b) the LLM call. At 1,000 concurrent the encoder moves behind a GPU batching service (millisecond-scale, ~100× headroom), with the index replicated read-only behind a load balancer — it is stateless, so replication is horizontal. Generation is the real constraint and is rate-limited: p50 is 1,557 ms paced but 7,296 ms sustained on free tier, a 4.7× gap that is quota, not inference, so production needs paid throughput plus a semantic cache. **Knowledge freshness** is already handled: embeddings are content-addressed on `(model, sha256(text))`, so re-indexing embeds only changed chunks.

Limitation before deployment: retrieval precision, via reranking. Recall is 0.889 while precision is 0.378 — the right chunk is usually retrieved but not ranked first, and the 6 grounded-but-wrong answers trace to weakly ranked context. The fix is a cross-encoder reranker over the top-50, scoring query and passage *jointly* rather than comparing independent vectors. Deliberately not implemented: on CPU it adds several hundred ms to a 168 ms retrieval path, so it only pays once the GPU tier above exists. A bigger embedding model is *not* the answer — measured, and it does not pay.